

Libtorch

Artificial Intelligence: The big umbrella. Any technique making computers smart.
(AI)

Machine Learning: Computers learning from data automatically. (Instead of hard coded rules)
(ML)

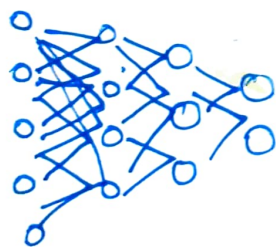
Deep Learning: A subset of ML using Neural Network with many layers.

3Blue 1Brown on What is a Neural Network?

→ Our visual cortex is able to take in any smudged or sloppy number written somewhere and able to understand it. How??

Neural Networks are inspired by brain.

Made up of several individual neurons.



} There are layers in between and each next layer is dependent on the previous one.

* A neural network breaks down an image or any element or object into patterns/edges.

* But we need a way, such that patterns can be picked up and brought in together

→ this is a parameter

Each neuron requires a weight, we take the activation and take a combined weighted sum.

$$w_1 a_1 + w_2 a_2 + w_3 a_3 + \dots + w_n a_n$$

↓
This alone would provide any random number

We need a function that would squish this between 0 and 1.

So,

$$\sigma(w_1 a_1 + w_2 a_2 + w_3 a_3 + \dots + w_n a_n)$$

↓
But we need not always force the NN only give a number above 0.

↙
For this we can add a bias, indicating, how high we need the activation number to be.

$$\sigma(w_1 a_1 + w_2 a_2 + w_3 a_3 + \dots + w_n a_n - b)$$

$\downarrow$
 "bias"

Learning $\rightarrow$ Getting the number and biases for prediction.

Compact way (formula) $\Rightarrow$

$$\sigma \left(\begin{bmatrix} w_{0,0} & w_{0,1} \\ w_{1,0} & w_{1,1} \\ \vdots & \vdots \\ w_{k,0} & w_{k,1} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_k \end{bmatrix} \right)$$

★ Each of the neuron is a function

$\hookrightarrow$ these all take data from previous network and give a value between 0 and 1.

Entire NN is one $\hookleftarrow$ big function.

★ Modern NN use ReLU $\rightarrow$ Rectified linear unit

$$\text{ReLU}(a) = \max(0, a)$$

Key Vocabulary To Know

- 1) Tensor → A multi-dimensional array of numbers. Like a spreadsheet but can have many dimensions.
- 2) Weights / Parameter → The "knobs" inside a neural network that get adjusted during learning.
- 3) Loss / Error → A number that measures how wrong the network's answer was. Lower = better.
- 4) Gradient → The direction and amount we should adjust each weight to reduce the loss.
- 5) Back propagation → The algorithm that computes gradients automatically by going backwards through the network.
- 6) Optimizer → A strategy for updating weights using gradients (eg. SGD, Adam).

7) Epoch

→ One complete pass through all training data.

What is LibTorch?

PyTorch's C++ twin — fast, powerful, production-ready

LibTorch → Official C++ distribution of PyTorch — it lets you write deep learning code in C++ instead of Python, with same powerful tensor operations, automatic differentiation, and NN modules.

Why do we need LibTorch?

↳ Because you need speed, you are shipping to robots/hardware, or you are embedding AI in a bigger C++ system.

Where is LibTorch used?

- self driving cars systems (processing camera feeds)
- Game Engines with AI characters
- ROS2 Robotic Systems

LibTorch Architecture

At the bottom level : ATen — the tensor library that does all the math.

On top of that Autograd — the automatic differentiation engine.

Torchscript and then C++ Frontend API

CMakeLists.txt File

cmake_minimum_required (VERSION 3.18 FATAL_ERROR)
project (libtorch-day1)

find_package(Torch ^{REQUIRED} Required)

set(CMAKE_CXX_STANDARD 17)

add_executable(app main.cpp)

target_link_libraries(app "\${TORCH_LIBRARIES}")

set_property(TARGET app PROPERTY CXX_STANDARD 17)

Basic Functioning code

```
#include <torch/torch.h>
```

```
#include <iostream>
```

```
int main () {
```

```
    // Creating a basic 3x3 tensor filled with  
    zeros
```

```
    torch::Tensor t = torch::zeros({3,3});
```

```
    std::cout << "A 3x3 zero tensor : " << std::endl;
```

```
    std::cout << t << std::endl;
```

```
    return 0;
```

```
}
```

1) `#include <torch/torch.h>` $\rightarrow$ The header file

2) `torch::` $\rightarrow$ namespace

$\hookrightarrow$ so, `torch::Tensor` can be shortened by writing

$\hookrightarrow$ `using namespace torch;` on top of the file

3) `torch::Tensor t` $\rightarrow$ We are declaring a variable named `t` of type `Tensor`.

4) `torch::zeros(...)` $\rightarrow$ Factory Function, PyTorch devs wrote a helper function called `zeros`, that build tensor for you.